Configuration & Secrets Runbook

Engineering Handbook · Internal · Owner: Platform Engineering

Purpose

This runbook is how we load configuration and secrets across every service (api-service, auth-service, worker-service, ingestion-service, scheduler, gateway). The recurring failure is a service that boots fine on a laptop but crashes on deploy with a `KeyError` because a required variable was never declared in the target environment. We fix this by validating all config once at startup, failing loudly with an actionable message, and sourcing secrets from Secret Manager rather than raw environment access scattered through the code.

Standards

Never read os.environ directly at call sites

Indexing `os.environ["DATABASE_URL"]` raises an uncaught `KeyError` at boot when the variable is missing, producing a stack trace with no guidance. Read through a single typed settings object that validates presence and type once, at startup.

```
from pydantic_settings import BaseSettings
from pydantic import Field

class Settings(BaseSettings):
   database_url: str = Field(..., alias="DATABASE_URL")   # required, no default
   redis_url: str = Field("redis://localhost:6379", alias="REDIS_URL")
   log_level: str = Field("INFO", alias="LOG_LEVEL")

settings = Settings()   # raises a clear ValidationError listing missing vars
```

For a lighter dependency, use `os.environ.get` with an explicit check so the error names the variable and how to set it:

```
url = os.environ.get("DATABASE_URL")
if not url:
   raise RuntimeError("DATABASE_URL is not set; add it to the deploy manifest/secrets")
```

Local .env is for development only

Load a `.env` with `python-dotenv` for local runs, but treat it as a convenience, not a source of truth. A variable that only lives in a developer's `.env` will be absent in production unless it is also declared in the deploy manifest.

```
from dotenv import load_dotenv
load_dotenv()   # no-op in prod where the file is absent
```

Secrets come from Secret Manager, not the image

Never bake secrets into the container, commit them, or pass them as plaintext build args. Reference them from Secret Manager and let the runtime inject them.

```
# Cloud Run: mount a secret as an env var
gcloud run deploy api-service \
 --set-secrets=DATABASE_URL=db-url:latest \
 --set-env-vars=LOG_LEVEL=INFO
```

- Grant the runtime service account `roles/secretmanager.secretAccessor` on the specific secret only.
- Pin to a version (`db-url:latest` is convenient but audit-sensitive; pin explicit versions for reproducibility).
- Keep config (non-sensitive) in env vars and secrets (sensitive) in Secret Manager; do not mix them.

Fail fast and log what is missing (not the value)

Validate the full config set on startup before the server accepts traffic. Log which keys were loaded, never their values, so a boot failure is diagnosable from logs alone.

Common failures and fixes

- `KeyError: 'DATABASE_URL'` at boot from reading `os.environ` directly → read with `os.environ.get` or a settings model, validate at startup with a clear error, and add `DATABASE_URL` to the deployment's env/secrets (BUG-1002).
- Works locally, fails in prod → the variable lived only in a local `.env`; declare every required var in the deploy manifest/secrets and validate presence at startup.
- Secret visible in logs or image → move it to Secret Manager, reference by name, and scrub values from log output.

Checklist

- All config is read through one settings object; no bare `os.environ[...]` at call sites.
- Required variables have no default and raise a named, actionable error when missing.
- Every required env var and secret is declared in the deploy manifest, not just the local `.env`.
- Secrets are stored in Secret Manager; the runtime service account has `secretAccessor` on each one.
- No secret is committed, baked into the image, or logged.
- Startup validation runs before the service reports ready; a missing var fails the deploy, not the first request.

Related incidents

BUG-1002